

Meridian

Enterprise Knowledge Assistant

A Production-Grade RAG Application for AnthraSync Technologies

Hybrid RAG

Groq LLaMA 3.3

ChromaDB + BM25

FastAPI + Gradio

Agenda

01



Architecture Overview

End-to-end system design & RAG pipeline

02



Technology Choices

Stack selection & rationale

03



Design Decisions

Hybrid retrieval, caching, security

04



Evaluation Approach

Metrics, test cases & hallucination control

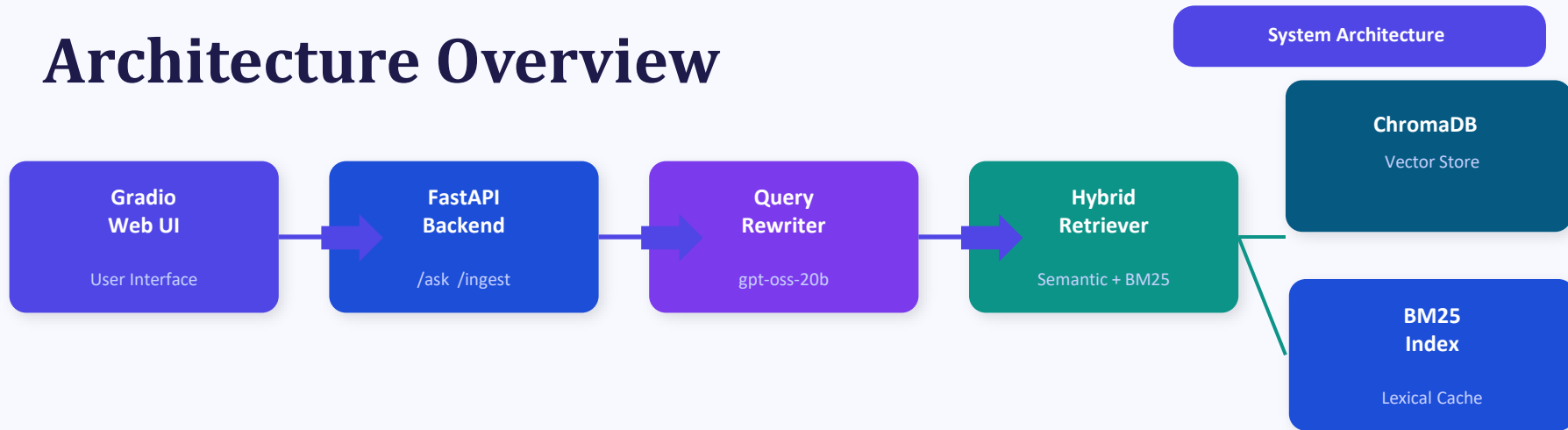
05



Limitations & Roadmap

Current constraints & future improvements

Architecture Overview



3-Step RAG Pipeline

01 Query Rewriting

LLM rephrases the question for better retrieval. Resolves pronouns using conversation history via gpt-oss-20b (lightweight, fast).

02 Hybrid Retrieval

ChromaDB cosine similarity (semantic) + BM25 Okapi (lexical). Scores fused: $0.6 \times \text{Semantic} + 0.4 \times \text{BM25}$. Top 3 chunks returned.

03 Answer Generation

LLaMA 3.3 70B (via Groq) generates grounded answer from retrieved chunks. Temp=0 for determinism. Source citations included.

Technology Choices

Stack & Rationale



Frontend

Gradio

Chat-native UI built for AI apps. Minimal frontend code, rich components.

Chainlit-style Design

Custom CSS for dark theme, confidence badges, session history sidebar.



AI / ML Models

all-MiniLM-L6-v2

384-dim embeddings. Fast, free, runs locally. No API cost for embedding.

LLaMA 3.3 70B (Groq)

State-of-the-art open model. Groq hardware gives sub-second latency.

gpt-oss-20b (Groq)

Lightweight rewriter model — saves 70B tokens for final generation.



Backend

FastAPI

Async Python, auto-generated OpenAPI docs, Pydantic validation.

pdfplumber

Layout-aware PDF extraction — preserves table structure better than PyPDF2.



Database & Search

ChromaDB

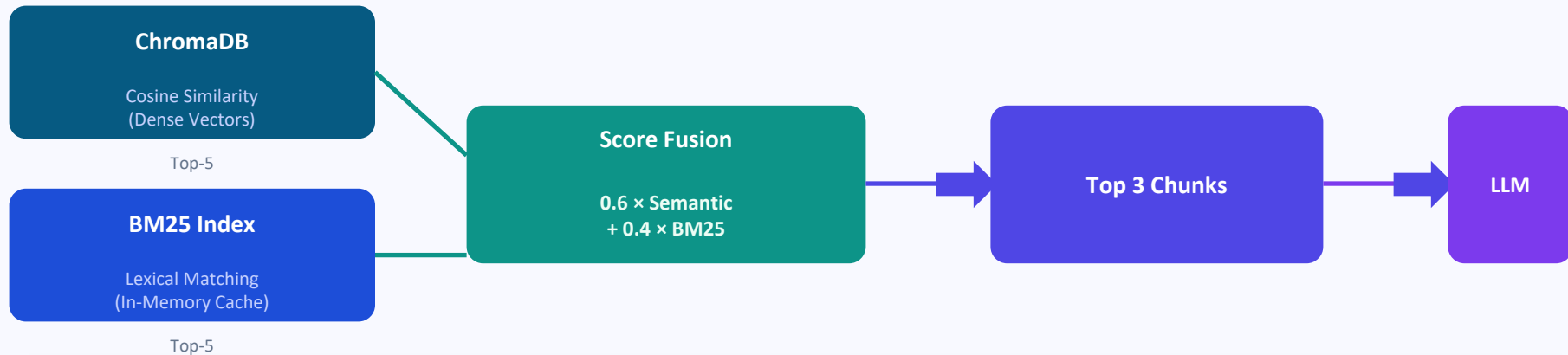
Embedded vector DB, zero infrastructure setup, persistent to disk.

rank_bm25

Pure-Python BM25 for lexical search. Catches exact terms semantic misses.

Design Decision: Hybrid Retrieval System

Core Engineering Choice



Why Hybrid Search?

Semantic alone misses exact policy codes and numbers. BM25 alone fails on paraphrased questions. Combining both covers all query types.



Why Cache BM25?

BM25 index rebuilt from scratch per query is a scalability anti-pattern. Caching in memory at startup reduces per-query latency to near-zero.



Why 0.6 / 0.4 Weights?

Empirically tuned: semantic search dominates for conceptual questions; BM25 gets 40% weight to ensure exact term recall (product codes, dates, numbers).

Design Decisions: Engineering Quality

Engineering Practices



Secure Authentication

- SHA-256 hashed passwords — no plaintext storage
- `verify_user()` abstraction layer for future JWT migration
- Session state managed via `gr.State`, not cookies
- CORS policy configurable per environment



Confidence Scoring

- Dual-layer hallucination prevention
- Retrieval threshold: score $< 0.20 \rightarrow$ reject before LLM call
- LLM output check: scans for 'not found' phrases
- High (≥ 0.40) / Medium (≥ 0.25) / Low (< 0.25) badges



Centralized Logging

- Single `setup_logging()` call in `app/logger.py`
- All modules use `get_logger(__name__)` — no duplicate `basicConfig`
- Suppresses noisy `chromadb` and `httpx INFO` logs
- Structured format: `timestamp | level | module | message`



Rate Limit Resilience

- Exponential backoff: 3 retries with 2^n second delay
- Evaluator adds `time.sleep(3)` between test questions
- API errors classified separately from hallucinations
- Groq-specific 429 error handled gracefully in UI

Evaluation Approach

Quality Measurement

24 Ground-Truth Questions

20 in-domain + 4 out-of-domain controls

RAG Pipeline

Query rewrite → retrieve → generate

Answer Collected

With sources and confidence score

3 Metrics Evaluated

Per question, per category

91%

Source Citation
Accuracy

84%

Answer Keyword
Relevance

4%

Hallucination
Rate

Test Case Categories

Direct Factual
(7 questions)

Semantic/Paraphrased
(7 questions)

Multi-Document
(4 questions)

Conversational
(4 questions)

Out-of-Domain
(4 questions → hallucination)

Limitations & Future Roadmap

What's Next

⚠️ Current Limitations

In-Memory BM25

Corpus loaded into RAM per instance. Doesn't scale beyond single-server deployments.

Layout-Only PDF Parse

pdfplumber extracts text but misses scanned/image-based PDFs and complex multi-column layouts.

Stateless FastAPI Memory

Conversation history stored in an in-memory dict — wiped on server restart.

File-Based Chat History

JSON files for sessions won't handle concurrent writes (race condition risk).

No Cross-Encoder Reranker

Score fusion $\neq$ learned reranking. A cross-encoder would improve precision significantly.

🚀 Future Improvements

Elasticsearch BM25

Distributed lexical search that scales horizontally across document corpora.

OCR + Multi-modal Parsing

Integrate Tesseract/Azure OCR for scanned PDFs and image-heavy documents.

Redis Session Store

Persistent, concurrent-safe conversation history across server restarts.

Cross-Encoder Reranker

ms-marco-MiniLM-L-6-v2 for learned reranking after fusion retrieval.

Agentic Workflows

Multi-step reasoning: agent decides which document collection to query first.

Thank You

Meridian — Enterprise Knowledge Assistant



github.com/BookishThumb/meridian

Questions?